

CTF 1

Instructor: Chethan Kamath

TAs: Shawn Koshy, Ravi Prakash, Priyanshu Singh, Harshit Somani and Reuben Toppo

Challenge 1: The Usual Suspects

A substitution cipher replaces every letter of the plaintext with another letter according to a fixed substitution table. For example, every occurrence of **a** might be replaced by **q**, every **b** by **x**, and so on.

Although this hides the actual letters, the substitution preserves an important property of the plaintext: the frequency and structure of the language.

You are given a ciphertext produced using a randomly generated monoalphabetic substitution cipher. The substitution key is not provided.

Your task is to recover the plaintext and find the flag.

You are provided with:

- `encrypt.py`: The encryption algorithm (with a dummy plaintext).
- `ciphertext.txt`: the encrypted plaintext.
- `solver.py`: an incomplete helper program for performing your analysis.

The `solver.py` file contains several functions marked `T0 D0`. Complete these functions and use the program to help recover the plaintext.

Hint: The plaintext is written in English. Start by examining the frequency of the letters in the ciphertext. Compare the observed frequencies with the typical frequencies of letters in the English language. Remember that the same substitution is used throughout the entire ciphertext. Word lengths, repeated letters, and common short words can also provide useful clues.

Submission details: You will submit the flag on CTFd platform, and the code (the completed `solver.py` file) and writeup on Moodle.

Challenge 2: Two-Time Pad

The one-time pad, as you have learnt, is perfectly secure. However, its perfect secrecy depends on the key being used exactly once. If the same key is reused, the resulting “two-time pad” is no longer secure as XORing the two ciphertexts gives you the XOR of the plaintexts, which is a lot of information about the two messages. In fact, if the messages are in natural language, it may even allow you to extract both the messages fully given their XOR. In this challenge, you’ll do exactly that.

`ciphertext1.enc` and `ciphertext2.enc` contain two one-time pad encrypted ciphertexts encrypted with the same key. One of them is a “flag” used in a Capture The Flag competition and the other one is a normal English sentence (hint: the flag starts with `cs409m{`, and the other english sentence is related to cryptography). Use the redundancy of the English language to figure out the entire flag. Feel free to look at `encrypt.py` to understand how to use the `strxor` function to XOR two byte objects in python.

Historical Note: *During WWII, the USSR made the mistake of using several one-time pad keys twice and the US intelligence agencies used this blunder to their advantage to decrypt the Soviet secret messages. Look up Venona Project if you're interested in the history!*

Submission details: You will submit the flag on CTFd platform, and the writeup on Moodle. You need not submit the code for this challenge.

Challenge 3: Faulty One-Time Pad Distinguishing Attack

True one-time pad encryption, as you have studied, is perfectly secure, that is, there is no way for any (potentially unbounded) adversary to distinguish between the encryption of string under one-time pad and a string sampled uniformly at random (of the appropriate length) with a non-zero advantage.

Bob, however, felt that the byte `0x00` in the key could potentially lead to security issues because it does not change the message character at all during the encryption. So instead of sampling each key-byte from the interval `[0x00, 0xff]`, he decided to instead sample the key-byte uniformly from the interval `[0x01, 0xff]`. Show that Bob's encryption scheme is no longer perfectly secure by distinguishing between the encryption of a string you provide and the encryption of a random message. (This is slightly different from IND-onetime security as defined in class, but it is also equivalent to perfect secrecy.)

You'll enter the hex-representation of your string (hint: the only thing that will matter is that your string is sufficiently long). Then you'll be presented with two strings: one which is a valid encryption of your string, and another which is the encryption of a random string, using a fresh random key (with the above modification). You need to identify which of the two is the encryption of your string. If you succeed to distinguish between the two 100 times in a row, you'll receive the flag!

Note that you will need to communicate with a server for this challenge. The template script provided to you handles the server communication for you using the `pwntools` library. You are only required to fill in the attack logic.

Submission details: You will submit the flag on CTFd platform, and the code (the completed `solution.template.py` file) and writeup on Moodle.

Challenge 4: One-Time-And

Alice, a cryptography expert at a massive company decides to encrypt confidential data using one time pad to distribute to the employees.

She uses several secret keys to encrypt her message, giving her several ciphers that she can distribute. These ciphers are stored in `ciphers.dat` in **hexadecimal** format.

However, she was working while eating popcorn and her buttery fingers slipped, leading to a typo: instead of the bitwise XOR (`^`) operator, she accidentally used the bitwise AND (`&`) operator.

You get your hands on the `ciphers.dat` file and the encryption algorithm `encrypt.py`. Use these files to exploit the vulnerability and decrypt Alice's confidential data!

Submission details: You will submit the flag on CTFd platform, and the code (the completed `solution.template.py` file) and writeup on Moodle.

Challenge 5: Poorly Random Generator

Johnny wants to send a TOP SECRET flag to Bobby! He doesn't want to send it all in one message, so he sends it one character at a time. He also fills each message with some filler content

(same for every message) to throw off eavesdroppers and make the message 16 bytes long. He encrypts each message using a computational one time pad:

$$s \leftarrow \{0, 1\}^{64}$$
$$c \leftarrow G(s) \oplus m$$

However, you notice that the PRG $G(s)$ that Johnny has used is faulty! You have also got your hands on the filler content that Johnny has added. Use this vulnerability to retrieve the TOP SECRET flag that Jonny tried to send. Each ciphertext is present in `ciphertexts.hex`, and the script Johnny used to encrypt is `encrypt.py`.

Submission details: You will submit the flag on CTFd platform, and the code (the completed `solution.template.py` file) and writeup on Moodle.

Challenge 6 (Bonus): Alphabet Soup

Recall the Vigenère cipher from class. Given a key $k = k_1 k_2 \dots k_L$ and a plaintext $p = p_1 p_2 \dots p_n$ over an alphabet of size N , we encrypt as

$$c_i = (p_i + k_{((i-1) \bmod L)+1}) \bmod N.$$

You have also seen how to break it, given enough ciphertext. First, use Kasiski's method or the Index of Coincidence to recover the key length. Then, since each key position gives its own Caesar shift, run a frequency-analysis attack on each column independently to recover the key. Standard treatments of both steps assume $N = 26$: the plaintext is stripped of case, spaces, digits, and punctuation, leaving only the letters A through Z.

Alice is not convinced this attack scales. She reasons that if she encrypts over the full set of 95 printable ASCII characters (0x20 to 0x7E), preserving spaces, case, digits, and punctuation, then Kasiski's repeated-substring counts will thin out and the IC signal will drown in the wider alphabet. She implements her scheme in `encryptor.py` and uses it to encrypt a long passage of English text with a passphrase drawn from the same 95-character alphabet. The flag is embedded somewhere in the plaintext.

You have been given `ciphertext.txt`. Show Alice she is wrong.

Hint. The attack you know still works here; the methods do not care about the alphabet size, only the numerical thresholds do. Somewhere in your attack you will need a frequency reference for English text over Alice's 95-character alphabet, and no such reference is sitting on the internet. You will have to build one yourself from any long English corpus (a Project Gutenberg book of comparable length is plenty). One warning: do *not* lower-case or strip punctuation from your corpus when you build the reference. Those characters are part of the alphabet now, and dropping them will poison it.

Hint (key length). To save you the trouble of detecting it yourself, Alice's key is 12 characters long. If you are curious, run the IC procedure and confirm it - you should see the average per-column IC jump sharply at $L = 12$.

Note on tooling. Nearly every online "Vigenère solver" assumes the 26-letter alphabet, strips everything else, and lower-cases its input. Passing this ciphertext through such a tool will give you garbage. Writing your own frequency-analysis code over the correct alphabet is the whole point of the exercise.

Think. Would this challenge be easier or harder if Alice had also stripped spaces before encrypting? What if her plaintext had been Python source code instead of English prose? (Include these in your writeup)

Submission details: You will submit the flag on CTFd platform, and the code (create your own solution file called `solution.py`, and output the flag) and writeup on Moodle.
